

Nama Kelompok: Kel 4 (Safenet Indonesia)

Kelas: TI-CCIT-6A

Anggota Kelompok : Aurio Hendrianoko Rajaa (2207412016)
Alya Yulhadi (2207412026)
Muhammad Nathan Sunarto (2207412024)
Rayhan Alfarizi (2207412001)

(Laporan NLP)

Abstract

Penelitian ini bertujuan untuk mengembangkan model analisis sentimen terhadap headline berita finansial dalam bahasa Inggris dan Indonesia menggunakan pendekatan berbasis Natural Language Processing (NLP) dan Machine Learning. Model yang dikembangkan mengintegrasikan teknik prapemrosesan teks, model LSTM dengan pendekatan TF-IDF, dan model XLMRoBERTa untuk mengklasifikasikan sentimen menjadi tiga kategori: positif, negatif, dan netral. Penelitian ini menggunakan dataset berita finansial dari file CSV dan menerapkan pelatihan serta pengujian untuk mengevaluasi performa model. Hasil menunjukkan akurasi rata-rata sebesar 0.69 untuk model LSTM dan performa yang lebih baik pada kelas netral dan positif dibandingkan kelas negatif. 1 Pendahuluan Analisis sentimen berita finansial merupakan salah satu aplikasi penting dalam Natural Language Processing (NLP) untuk mendukung pengambilan keputusan di pasar keuangan. Dengan meningkatnya volume berita finansial, baik dalam bahasa Inggris maupun Indonesia, diperlukan model prediktif yang dapat secara akurat mengklasifikasikan sentimen (positif, negatif, atau netral) dari headline berita. Penelitian ini mengembangkan dua pendekatan utama: model berbasis LSTM dengan TF-IDF untuk berita dalam bahasa Inggris dan model XLMRoBERTa untuk berita dalam bahasa Indonesia.

1. Tujuan Penelitian

Tujuan penelitian ini adalah:

- Mengembangkan model analisis sentimen berbasis LSTM dan XLM-RoBERTa untuk headline berita finansial.
- Melakukan pra-pemrosesan teks yang optimal untuk mempertahankan konteks finansial.
- Mengevaluasi performa model menggunakan metrik akurasi, precision, recall, dan F1-score.
- Membandingkan efektivitas model dalam menganalisis sentimen berita dalam bahasa Inggris dan Indonesia.

1.2 Manfaat Penelitian Hasil penelitian ini diharapkan dapat:

- Membantu investor dan analis keuangan dalam memahami sentimen pasar berdasarkan berita.
- Menyediakan model yang andal untuk analisis otomatis sentimen berita finansial.
- Memberikan wawasan tentang penerapan NLP dalam konteks keuangan multibahasa.

2. Gambaran Umum Sistem

Sistem analisis sentimen berita finansial yang dikembangkan terdiri dari tiga komponen utama: pra-pemrosesan teks, pembangunan model, dan pengujian. Sistem ini dirancang untuk menangani headline berita finansial dalam bahasa Inggris dan Indonesia dengan pendekatan yang berbeda untuk masing-masing bahasa.

3. Pengembangan Model Financial Sentiment Analysis

Bagian ini menjelaskan secara rinci proses pengembangan model untuk melakukan *analysis sentiment* terhadap sebuah *headline* berita finansial. Pendekatan yang digunakan mengintegrasikan teknik *Text Preprocessing* untuk membersihkan dan menormalkan data dan *Machine Learning* untuk pembangunan dan pengoptimalan model prediktif.

Proses ini mencakup teks *preprocessing*, *modelling* dan juga *testing*. Setiap tahapan diuraikan secara sistematis untuk memastikan bahwa model yang dihasilkan akurat, andal, dan relevan dalam konteks analisis sentiment berita finansial.

3.1 Sentiment Analysis Berita Finansial Bahasa inggris

Dataset yang digunakan dalam penelitian ini merupakan kumpulan *headline* berita finansial yang mencakup informasi relevan untuk di analisis sentimentnya. *Dataset* ini bersumber dari *file CSV* berjudul *data.csv*, yang berisi 5843 baris dan 2 kolom, yaitu:

1. Sentence
2. Sentiment

Fitur-fitur ini dipilih dikarenakan relevansinya dalam mendeteksi sebuah sentimen dari judul berita finansial yang ada

3.2 Configuration

Tahap ini mencakup konfigurasi parameter pada model *TF-IDF* dan *LSTM* yang di gunakan oleh project analisis sentimen ini. Konfigurasi parameter itu sendiri di bagi menjadi berikut:

1. Konfigurasi parameter teks processing, paths dan juga model

```
# Paths
DATA_PATH = "data.csv"
MODEL_SAVE_PATH = "financial_sentiment_model_unified.pkl"

# Text Processing
MAX_LENGTH = 150
MAX_VOCAB_SIZE = 20000

# Model Parameters
SEED = 42
TEST_SIZE = 0.15
VAL_SIZE = 0.1
BATCH_SIZE = 32
```

Pada tahap ini, konfigurasi diterapkan untuk mendukung pemrosesan data dan pelatihan model analisis sentimen yang diintegrasikan dengan prediksi harga saham.

Konfigurasi *Paths* mencakup *DATA_PATH* yang ditetapkan pada *"/content/drive/MyDrive/Colab Notebooks/stock-price-prediction-sentiment-analysis/data.csv"* untuk menentukan lokasi *dataset* di penyimpanan *cloud*, dan *MODEL_SAVE_PATH* sebagai *"financial_sentiment_model_unified.pkl"* untuk menyimpan model terlatih dalam satu *file unified*.

Konfigurasi *Text Processing* dan *Model Parameters* meliputi *MAX_LENGTH* sebesar 150 untuk menangkap lebih banyak konteks teks, *MAX_VOCAB_SIZE* sebesar 20000 untuk memperluas cakupan vokabulari, *SEED* sebesar 42 untuk memastikan konsistensi hasil, *TEST_SIZE* sebesar 0.15 dan *VAL_SIZE* sebesar 0.1 untuk pembagian data uji dan validasi, serta *BATCH_SIZE* sebesar 64 untuk meningkatkan stabilitas pelatihan dengan memproses lebih banyak data per iterasi, sehingga model dapat menghasilkan prediksi sentimen yang lebih akurat dan *robust*.

2. Konfigurasi *LSTM* dan *TF-IDF*

```
# LSTM Parameters (optimized)
EMBEDDING_DIM = 200
LSTM_UNITS = 64
DROPOUT_RATE = 0.2
SPATIAL_DROPOUT = 0.15
EPOCHS = 20
LEARNING_RATE = 0.001

# TF-IDF configuration
TFIDF_MAX_FEATURES = 2000
TFIDF_NGRAM_RANGE = (1, 1)
CLASS_WEIGHTS = {0: 3.0, 1: 3.0, 2: 2.0}
```

Pada tahap ini, konfigurasi parameter diimplementasikan untuk meningkatkan performa model analisis sentimen dengan pendekatan yang lebih kompleks. Konfigurasi *LSTM* mencakup *EMBEDDING_DIM* sebesar 200 untuk representasi kata yang lebih kaya, *LSTM_UNITS* ditingkatkan menjadi 128 untuk kapasitas pemrosesan yang lebih besar, *DROPOUT_RATE* sebesar 0.3 dan *SPATIAL_DROPOUT* sebesar 0.15 untuk mengoptimalkan pencegahan *overfitting*, *EPOCHS* ditingkatkan menjadi 50 untuk konvergensi yang lebih baik, serta *LEARNING_RATE* sebesar 0.0001 yang akan dikurangi setengahnya selama pelatihan untuk stabilitas yang lebih tinggi.

Konfigurasi *TF-IDF* meliputi *TFIDF_MAX_FEATURES* yang ditingkatkan menjadi 5000 untuk menangkap lebih banyak fitur, *TFIDF_NGRAM_RANGE* diperluas menjadi (1, 2) untuk memasukkan unigram dan bigram guna meningkatkan konteks, serta *CLASS_WEIGHTS* dengan bobot {0: 3.0, 1: 3.0, 2: 2.0} untuk menangani ketidakseimbangan kelas, sehingga model dapat menghasilkan analisis sentimen yang lebih akurat dan mendalam.

3.3 Teks *Pre-Processing*

Pre-processing yang dilakukan di bagian analisis sentimen ini dilakukan dengan membersihkan dan menormalkan data teks dengan langkah-langkah seperti: melestarikan frasa finansial (seperti "*interest rate*"), mengganti simbol khusus (misalnya

\$ menjadi "*dollar*"), menormalisasi angka ("15%" menjadi "15 *percent*"), menangani negasi ("*not profitable*" menjadi "*neg_profitable*"), membersihkan teks (menghapus URL/HTML), melakukan tokenisasi dan lematisasi ("*running*" → "*run*"), serta menghapus *stopwords* kecuali kata negasi atau istilah finansial penting. *Function* pada *pre-prososing* data yang dilakukan sebagai berikut:

1. *init(self)*

Fungsi inisialisasi ini bertugas mengatur komponen utama untuk pra-pemrosesan teks finansial. Fungsi ini menginisialisasi *lemmatizer* (untuk mereduksi kata ke bentuk dasar), *tokenizer* (untuk memisahkan kata dengan pola *regex*), dan daftar *stopword* yang telah dimodifikasi untuk mempertahankan kata-kata finansial penting (misalnya, "*up*", "*down*"). Selain itu, fungsi ini mendefinisikan kamus *token* khusus (seperti "\$" menjadi "*dollar*"), pola *regex* untuk mendeteksi elemen finansial (*tickers*, mata uang) daftar frasa finansial yang harus dipertahankan, serta daftar istilah positif, negatif, negasi, dan pengubah intensitas untuk analisis sentimen.

2. *_preserve_financial_phrases(self, text: str) -> str*

Fungsi ini berfungsi untuk melindungi frasa finansial multi-kata (seperti "*market cap*" atau "*earnings per share*") agar tidak terpecah selama pra-pemrosesan. Fungsi ini mengganti spasi dalam frasa tersebut dengan garis bawah (*underscore*) untuk memastikan frasa tetap utuh, misalnya mengubah "*market cap*" menjadi "*market_cap*". Hal ini penting untuk menjaga makna konteks finansial dalam teks.

3. *_handle_special_tokens(self, text: str) -> str*

Fungsi ini memproses *token* khusus terkait finansial, seperti simbol mata uang ("\$" menjadi "*dollar*", "€" menjadi "*euro*") dan istilah singkatan (seperti "*k*" menjadi "*thousand*"). Fungsi ini mengganti *token* tersebut dengan kata pengganti yang lebih deskriptif untuk memudahkan pemrosesan teks sambil mempertahankan makna finansial.

4. *_process_numbers(self, text: str) -> str*

Fungsi ini menormalkan ekspresi numerik dalam teks, seperti persentase (misalnya, "15%" menjadi "15 percent") dan jumlah uang (misalnya, "\$500" menjadi "dollar 500"). Selain itu, fungsi ini menangani singkatan seperti "K" (ribu), "M" (juta), "B" (miliar), dan "T" (triliun) untuk memastikan representasi numerik yang konsisten dan mudah dipahami oleh model.

5. *handle_negations(self, text: str) -> str*

Fungsi ini menangani kata-kata negasi (seperti "not", "never") yang dapat membalikkan sentimen dalam teks. Fungsi ini mencari kata negasi dan memeriksa hingga lima kata berikutnya untuk menemukan istilah sentimen positif atau negatif.

Jika ditemukan, istilah tersebut ditandai dengan prefiks "neg" (misalnya, "profit" menjadi "neg_profit") untuk menunjukkan bahwa sentimennya telah dibalik, sehingga membantu model memahami konteks sentimen dengan lebih baik.

6. *extract_sentiment_features(self, text: str) -> dict*

Fungsi ini mengekstrak fitur sentimen dari teks untuk analisis lebih lanjut. Fungsi ini menghitung jumlah istilah positif dan negatif, memperhitungkan negasi (misalnya, "not profit" dihitung sebagai sentimen negatif), dan mengevaluasi pengubah intensitas (seperti "very" atau "extremely") untuk menambah bobot sentimen. Selain itu, fungsi ini menghitung metrik tambahan seperti keberadaan tanda seru, jumlah tanda tanya, kata-kata dalam huruf kapital, dan indikator pergerakan harga (seperti "increase" atau "drop").

Hasilnya adalah kamus yang berisi skor sentimen, intensitas, polaritas, dan fitur tambahan untuk analisis mendalam.

7. *preprocess(self, text: str) -> str*

Fungsi ini adalah *pipeline* utama untuk pra-pemrosesan teks. Fungsi ini menjalankan serangkaian langkah: mengubah teks ke huruf kecil, melindungi frasa finansial, menangani *token* khusus, memproses ekspresi numerik, menangani negasi, menghapus *URL* dan *tag HTML*, melakukan tokenisasi, menerapkan lematisasi, dan menghapus *stopword* sambil mempertahankan istilah negasi dan finansial penting. Hasilnya adalah teks yang telah dibersihkan dan dioptimalkan untuk analisis sentimen, dengan fokus pada konteks finansial.

3.4 Pembangunan Model LSTM dan Pengoptimisasi Model

Proses pembangunan model *LSTM* ini menggunakan tiga cabang input: Cabang Teks dengan *Embedding* dan *Bidirectional LSTM* untuk memproses konteks temporal. Cabang *TF-IDF* dengan lapisan *dense* untuk mengekstrak bobot kata penting, dan cabang fitur sentimen yang memproses fitur numerik. Ketiganya digabungkan melalui *concatenate*, lalu diproses oleh lapisan *dense* untuk klasifikasi akhir (negatif/netral/positif). *Function* yang di gunakan pada pembangunan dan pengoptimisasian model sebagai berikut:

1. *__init__(self)*

Menginisialisasi model *FinancialSentimentLSTM* dengan membuat *instance FinancialTextPreprocessor*, menyiapkan *tokenizer*, *vectorizer TF-IDF*, dan model *LSTM*. Menetapkan parameter seperti panjang maksimum teks dan ukuran vokabulari dari konfigurasi, serta mendefinisikan aturan pasca-pemrosesan untuk prediksi sentimen.

2. *_build_advanced_lstm_model(self, vocab_size: int, num_classes: int, tfidf_dim: int) -> Model*

Membangun arsitektur *LSTM bidirectional* dengan tiga cabang input: teks (*embedding* dan *LSTM*), fitur *TF-IDF* (lapisan *dense*), dan fitur sentimen (lapisan *dense*). Menggabungkan cabang-cabang ini untuk klasifikasi sentimen dengan *optimizer Adam* dan *loss sparse categorical crossentropy*.

3. *fit(self, X: Union[pd.Series, List[str]], y: pd.Series)*

Melatih model dengan memproses teks input melalui pra-pemrosesan, ekstraksi fitur sentimen, tokenisasi, dan vektorisasi *TF-IDF*. Membagi data menjadi set pelatihan dan validasi, menerapkan bobot kelas untuk ketidakseimbangan, dan melatih model dengan *callback* seperti *early stopping* dan *checkpoint*.

4. *predict_proba(self, X: Union[pd.Series, List[str]]) -> np.ndarray*

Menghasilkan probabilitas prediksi untuk input teks dengan memprosesnya melalui pra-pemrosesan, ekstraksi fitur sentimen, tokenisasi, dan vektorisasi *TF-IDF*. Model kemudian menghasilkan probabilitas untuk setiap kelas sentimen.

5. *predict(self, X: Union[pd.Series, List[str]]) -> np.ndarray*

Menghasilkan prediksi kelas sentimen berdasarkan probabilitas dari *predict_proba*. Menerapkan aturan pasca-pemrosesan untuk mengubah prediksi netral menjadi positif atau negatif jika kata kunci tertentu ditemukan dan kepercayaan di bawah ambang batas.

6. *predict_single_text(self, text: str) -> dict*

Memproses satu teks untuk menghasilkan analisis sentimen mendetail, termasuk sentimen prediksi, kepercayaan, probabilitas per kelas, dan waktu pemrosesan. Menerapkan aturan pasca-pemrosesan untuk meningkatkan akurasi prediksi.

7. *save_unified(self, filepath='financial_sentiment_model_unified.pkl')*

Menyimpan seluruh komponen model (*preprocessor*, *tokenizer*, *vectorizer TF-IDF*, dan model *Keras*) ke dalam satu *file pickle*, dengan model *Keras* disimpan terpisah dalam *format .h5* untuk penggunaan *API* yang lebih mudah.

8. `load_unified(cls, filepath='financial_sentiment_model_unified.pkl')`

Memuat model lengkap dari *file pickle* dan *file .h5*, mengembalikan *instance FinancialSentimentLSTM* dengan semua komponen yang telah diinisialisasi untuk prediksi.

9. `run_training_pipeline(data_path=Config.DATA_PATH)`

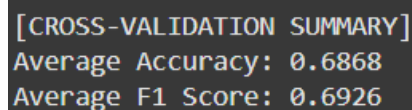
Menjalankan *pipeline* pelatihan dengan validasi silang *stratified k-fold*. Memuat data, memetakan label sentimen, melatih model pada setiap *fold*, mengevaluasi metrik seperti akurasi dan *F1*, serta menyimpan model terbaik berdasarkan skor *F3*.

10. `test_prediction(model, texts)`

Menguji model dengan teks sampel, menghasilkan prediksi sentimen, kepercayaan, dan distribusi probabilitas. Menampilkan visualisasi probabilitas dan statistik seperti waktu prediksi rata-rata serta distribusi kelas prediksi.

3.5 Visualisasi

Tahap ini dilakukan untuk menunjukkan hasil dari pengerjaan model LSTM. Visualisasi dari model yang di gunakan sebagai berikut:



```
[CROSS-VALIDATION SUMMARY]
Average Accuracy: 0.6868
Average F1 Score: 0.6926
```

Gambar 3.1 Visualisasi Model

Hasil dari evaluasi *cross validation* yang didapatkan rata-rata akurasinya 0.69 dan rata-rata dari *f1 score* adalah 0.69.

	precision	recall	f1-score	support
negative	0.56	0.87	0.68	118
neutral	0.91	0.80	0.85	476
positive	0.92	0.88	0.90	282
accuracy			0.84	876
macro avg	0.80	0.85	0.81	876
weighted avg	0.87	0.84	0.85	876

Gambar 3.2 Hasil Cross Validation

Tabel klasifikasi menunjukkan metrik performa model sentimen finansial: negatif (*precision* 0.56, *recall* 0.87, *f1* 0.68, 118 sampel), netral (0.91, 0.80, 0.85, 476 sampel), positif (0.92, 0.88, 0.90, 282 sampel). Akurasi 0.84 (876 sampel), dengan *macro average* (0.80, 0.85, 0.81) dan *weighted average* (0.87, 0.84, 0.85), menunjukkan kelas netral dan positif lebih akurat dibanding negatif.

3.6 Model Sentiment Analysis Berita Finansial Bahasa Indonesia

Bagian ini menjelaskan secara rinci proses pengembangan model untuk melakukan analisis sentiment terhadap sebuah *headline* berita finansial. Pendekatan yang digunakan mengintegrasikan teknik *Text Preprocessing* untuk membersihkan dan menormalkan data dan *Natural language Processing* untuk pembangunan dan pengoptimalan model prediktif.

Proses ini mencakup teks *preprocessing*, *modelling* dan juga *testing*. Setiap tahapan diuraikan secara sistematis untuk memastikan bahwa model yang dihasilkan akurat, andal, dan relevan dalam konteks analisis sentiment berita finansial. Untuk susunan *logiknya* sebagai berikut:

1. Pengaturan Awal (Print dan Device Check)

Program dimulai dengan menampilkan *header* untuk menandai awal pelatihan dan mencatat waktu mulai menggunakan *datetime.now()*. Kemudian, memeriksa ketersediaan *GPU* dengan *torch.cuda.is_available()* dan menetapkan perangkat (*device*) ke *GPU* jika tersedia, atau *CPU* jika tidak. Jika

GPU aktif, informasi nama dan memori ditampilkan untuk memastikan sumber daya yang digunakan optimal.

2. Memuat *Dataset* (Data Loading)

Menggunakan *pd.read_csv()* untuk memuat *dataset* dari "*translated_data.csv*", mencatat waktu pemuatan dengan *time.time()*, dan menampilkan detail seperti ukuran dan kolom *dataset*. Ini memastikan data berhasil dimuat dan memberikan gambaran awal tentang struktur data.

3. *Encoding* Label Sentimen (Label Encoding)

Menginisialisasi *LabelEncoder()* untuk mengonversi label sentimen (negatif, netral, positif) menjadi nilai numerik, mencatat waktu proses. Pemetaan label disimpan dalam *label_mapping* untuk referensi, memastikan label siap digunakan dalam model *machine learning*.

4. Pemisahan Data (Data Splitting)

Menggunakan *train_test_split()* untuk membagi data teks dan label menjadi 80% pelatihan dan 20% validasi dengan *random_state=42* untuk reproduktibilitas, mencatat waktu pemisahan. Jumlah sampel untuk setiap bagian ditampilkan untuk verifikasi.

5. Distribusi Data Pelatihan dan Validasi

Menghitung distribusi kelas sentimen menggunakan *value_counts()* pada label yang diencode, kemudian menampilkan persentase setiap kelas untuk pelatihan dan validasi. Ini membantu memeriksa keseimbangan data sebelum pelatihan.

6. Memuat *Tokenizer XLMRoBERTa* (*Tokenizer Loading*)

Memuat	tokenizer	dari	model	pralatih
"cardiffnlp/twitterxlmrobertabasesentiment"				menggunakan

XLMLRobertaTokenizer.from_pretrained(), mencatat waktu pemuatan. Ukuran kosakata ditampilkan untuk memverifikasi *tokenizer* siap digunakan.

7. Tokenisasi Teks (*Tokenizing Texts*)

Fungsi *tokenize_texts()* mengonversi teks menjadi *token* dengan panjang maksimum 256, *truncation*, dan *padding*, mengembalikan *tensor PyTorch*. Blok kode menerapkannya pada data pelatihan dan validasi secara terpisah, mencatat waktu dan menampilkan bentuk input untuk memastikan tokenisasi berhasil.

8. Membuat *Dataset PyTorch* (*Dataset Creation*)

Menggabungkan input *IDs*, *attention mask*, dan label menjadi *TensorDataset* untuk pelatihan dan validasi, mencatat waktu pembuatan. Ini mempersiapkan data dalam format yang kompatibel dengan *PyTorch* untuk pelatihan model.

9. Pengaturan *DataLoader* (*DataLoader Setup*)

Membuat *DataLoader* dengan *batch size* 16, menggunakan *RandomSampler* untuk pelatihan dan *SequentialSampler* untuk validasi, mencatat waktu setup. Jumlah *batch* ditampilkan untuk memastikan data siap diolah secara *berbatch*.

10. Memuat Model XLMRoBERTa (Model Loading)

Memuat model dari "*cardiffnlp/twitterxlmrobertabasesentiment*" dengan 3 kelas sentimen menggunakan *XLMLRobertaForSequenceClassification.from_pretrained()*, memindahkannya ke perangkat dengan *model.to(device)*, dan mencatat waktu pemuatan. Jumlah parameter ditampilkan untuk mengevaluasi kompleksitas model.

11. Pengaturan *Optimizer* (*Optimizer Setup*)

Menginisialisasi *optimizer AdamW* dengan *learning rate* $2e5$ untuk parameter model, memastikan proses optimasi siap untuk menyesuaikan bobot selama pelatihan.

12. Loop Pelatihan (*Training Loop*)

Melatih model selama 3 *epoch*, setiap *epoch* mengatur *mode train*, menghitung *loss* per *batch*, melakukan *backpropagation* dengan *loss.backward()* dan *clipping gradien*, serta memperbarui bobot dengan *optimizer.step()*. Progres ditampilkan setiap 50 *batch*, dan waktu serta rata-rata *loss* dihitung per *epoch*.

13. Evaluasi Model

Mengatur *mode eval*, memproses data validasi dalam *batch* tanpa gradien (*torch.no_grad()*), menghitung akurasi secara bertahap dengan membandingkan prediksi dan label, dan menyimpan semua prediksi serta label untuk analisis mendalam, dengan waktu evaluasi dicatat.

14. Laporan Klasifikasi (*Classification Report*)

Menggunakan *classification_report()* untuk menghitung metrik seperti *precision*, *recall*, dan *f1score* berdasarkan prediksi dan label, serta *confusion_matrix()* untuk menampilkan matriks kebingungan, memberikan gambaran performa per kelas.

15. Akurasi Per Kelas

Menghitung akurasi per kelas dari diagonal matriks kebingungan, membagi jumlah prediksi benar dengan total sampel per kelas, dan menampilkan hasil untuk mengevaluasi performa spesifik setiap sentimen.

16. Menyimpan Model

Menyimpan state model, *tokenizer*, label *encoder*, konfigurasi, dan info pelatihan ke dalam kamus, lalu menyimpannya ke *file pickle*

"financial_sentiment_model.pkl" menggunakan *pickle.dump()*, mencatat waktu dan ukuran *file*.